

Práctica 3: Desarrollo y Pruebas Unitarias de un Sistema de Reservas Vacacionales en Flutter

Marcos García Álvarez Germán Delgado Fernández
Javier Miñan Molina

29 de abril de 2026

Índice

1. Introducción	2
2. Diseño del Software	2
2.1. Patrones de Diseño Utilizados	2
2.2. Reglas de Negocio y Validaciones	2
3. Diagrama UML	2
4. Planificación y Ejecución de Pruebas	2
5. Interfaz de Usuario	3
6. Capturas de Ejecución	4
7. Conclusiones	5
8. Resumen	5

1. Introducción

El proyecto consiste en una aplicación Flutter para determinar el coste de paquetes vacacionales. El sistema permite gestionar elementos individuales (Vuelos y Hoteles) y agruparlos de forma recursiva. El reto principal reside en la tarificación dinámica, donde el coste final depende de políticas seleccionadas en tiempo de ejecución, manteniendo el encapsulamiento estricto de valores internos (recargos y multiplicadores).

2. Diseño del Software

2.1. Patrones de Diseño Utilizados

Se han aplicado dos patrones de diseño fundamentales:

- **Patrón Composite:** La interfaz `ServicioTuristico` permite tratar de forma uniforme tanto a servicios individuales (Vuelo/Hotel) como a agrupaciones (`Paquete`). El método `getPrecio()` se propaga de forma recursiva.
- **Patrón Strategy:** El cálculo del precio se delega a interfaces de política (`PoliticaVuelo`, `PoliticaHotel`). Esto permite que un mismo servicio cambie su comportamiento de coste dinámicamente sin modificar su clase original.

2.2. Reglas de Negocio y Validaciones

Se han implementado mecanismos de control de errores mediante excepciones en Dart:

- **Vuelo:** El constructor lanza un `ArgumentError` si el precio base es negativo.
- **Hotel:** Se impide la creación de estancias con cero o menos noches mediante validación en el constructor.

3. Diagrama UML

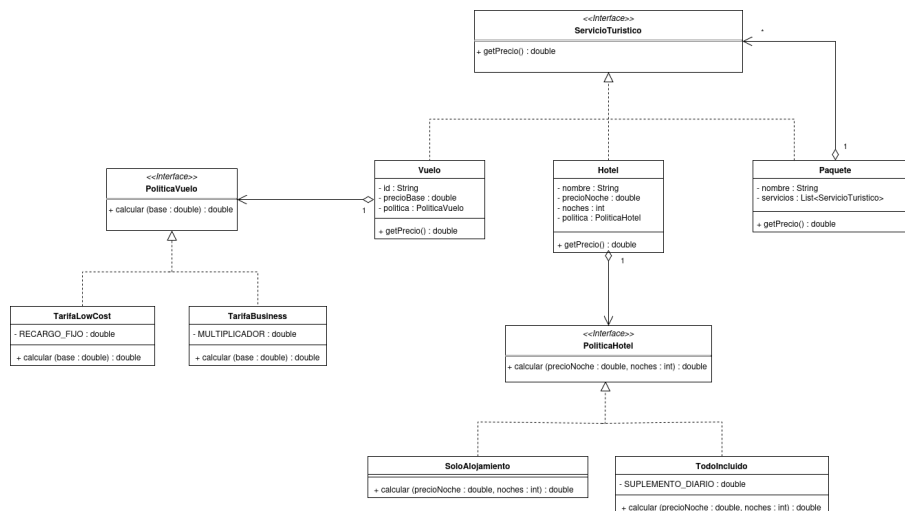


Figura 1: Diagrama UML.

4. Planificación y Ejecución de Pruebas

Se han desarrollado 12 tests unitarios organizados en tres grupos lógicos para maximizar la cobertura de errores.

Grupo	Tests	Objetivo de la prueba
Políticas	4	Verificar cálculos matemáticos de LowCost, Business, SoloAlojamiento y TodoIncluido.
Servicios	4	Validar el lanzamiento de excepciones ante datos erróneos y la correcta delegación del cálculo.
Paquetes	4	Comprobar la suma recursiva, el estado inicial vacío y la reactividad al cambiar políticas.

Cuadro 1: Resumen del plan de pruebas unitarias.

5. Interfaz de Usuario

La interfaz se ha modernizado para incluir:

- **Pantalla de Inicio:** Bienvenida y creación del paquete con nombre personalizado.
- **Diálogos de Configuración:** `AlertDialog` para introducir precios y noches de forma dinámica.
- **Gestión de Errores:** Uso de `SnackBar` para notificar al usuario si intenta introducir valores inválidos (negativos o cero).

6. Capturas de Ejecución

A continuación se presentan las evidencias del sistema en funcionamiento y los tests superados.

```
javi@javi-HP-Envy-x360-2-in-1-Laptop-14-fa0xxx:~/asa$ flutter test
00:08 +12: All tests passed!
```

Figura 2: Ejecución de los 12 tests unitarios superados correctamente.

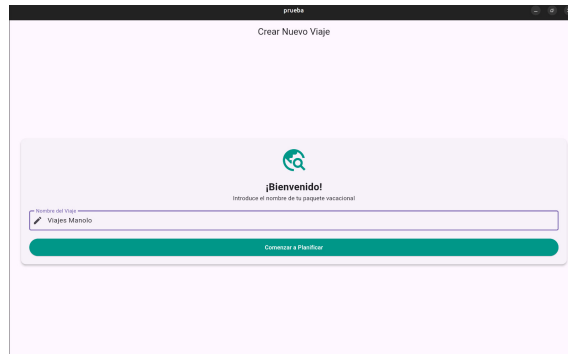


Figura 3: Pantalla de creación del paquete vacacional.

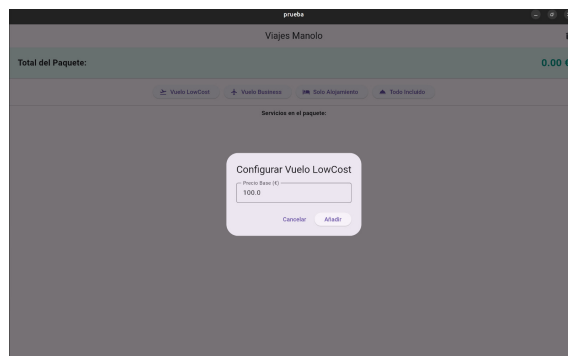


Figura 4: Añadiendo vuelo.

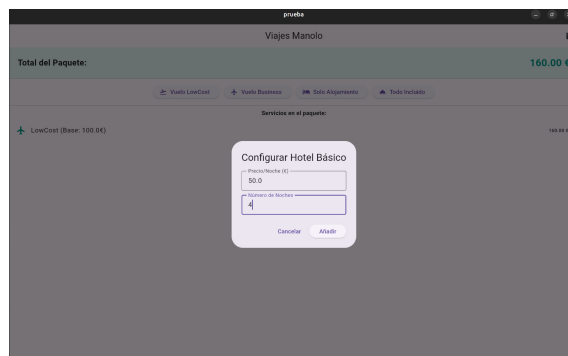


Figura 5: Añadiendo hotel.

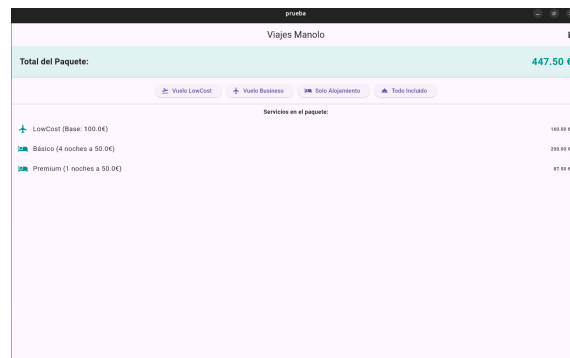


Figura 6: Paquete creado.

7. Conclusiones

La implementación demuestra que el uso de patrones de diseño no solo facilita la organización del código, sino que hace que las pruebas sean triviales de implementar. La validación temprana en los constructores protege la integridad del sistema, mientras que el patrón Strategy ofrece la flexibilidad necesaria para un sistema de reservas real.

8. Resumen

Esta memoria técnica describe la implementación de una plataforma de reservas que utiliza patrones de diseño estructurales para gestionar servicios turísticos. Se detalla la jerarquía de clases, la lógica de tarificación dinámica mediante políticas y un plan de pruebas unitarias que garantiza la integridad de los cálculos y las restricciones de negocio.